

```

# Full-Stack Project Blueprint & Prompt Sheet

### MERN + TypeScript — Industry Agnostic

> Version: 2.2
> Document snapshot: Practices are maintained over time — dependency numbers in
this file are not authoritative.
> Audience: Solo developers, AI coding assistants (Cursor, Copilot, Claude),
team developers, non-technical stakeholders
> Purpose: Copy-paste prompts, security checklists, folder structure, and API
documentation standards for any full-stack project — not tied to any specific
industry.

---

## ⚠ Version Safety Rule (Read First)

> Never copy version numbers from this document (or from memory) into
`package.json`. Old pins stay vulnerable; docs go stale the day they ship.

### For humans

1. Open [npmjs.com](https://www.npmjs.com/) for each package — confirm latest
and read the Security / advisory links if shown.
2. Prefer the newest patched release on a supported major line — not an old pin
“because the tutorial said so.”
3. After install: `npm audit` and fix high/critical before shipping.

```bash
npm show <package-name> version # latest published
npm view <package-name> time.modified # how fresh “latest” is
npm audit # installed tree
```

### For AI assistants (mandatory)

Before you write or edit `package.json`, `package-lock.json`, or recommend
install commands:

1. Web search (use your built-in web search tool when available) for each
non-trivial dependency, for example:
  - `npm <package-name> latest version`
  - ` npm security advisory` or ` CVE`
  For Node.js, search e.g. `Node.js LTS current release` and align with
Active LTS, not EOL versions.

```

2. Cross-check with `**`npm show <package> version`**` in the terminal when the environment allows – search can lag; npm registry is ground truth for the semver string.
3. If search turns up an **unpatched CVE** on ``latest``, search again for **patched version** or mitigation (or choose an alternative package).
4. In your reply, **briefly state** what you verified (e.g. “searched + ``npm show express version`` → using ``^5.x.y``”) – do not silently invent versions.
5. Use **range** in ``package.json`` for application deps unless you have a documented reason to pin an exact version – then pin **to a verified good release**, not an old one.

No hardcoded version table in this file – the list below is only **names to verify** (not versions):

| Package (verify each) | What to search / check |
|--|--|
| <code>`express`</code> | Latest stable major line; Express / Node compatibility; advisories |
| <code>`mongoose`</code> | Latest stable; compatibility with your MongoDB driver / Atlas |
| <code>`zod`</code> | Latest stable; note if a new major is in beta – don’t adopt beta for production without intent |
| <code>`jsonwebtoken`</code> , <code>`bcryptjs`</code> , <code>`express-rate-limit`</code> , <code>`helmet`</code> , <code>`cors`</code> , <code>`socket.io`</code> | Same pattern: npm latest + advisory search |

Repeat for **every** dependency you add (including ``react``, ``vite``, ``@sentry/node``, etc.).

Table of Contents

1. [Tech Stack](#1-tech-stack)
2. [Repository Structure](#2-repository-structure)
3. [Environment Variables](#3-environment-variables)
4. [Backend Architecture](#4-backend-architecture)
5. [API Documentation Standard](#5-api-documentation-standard)
6. [Backend Security Checklist](#6-backend-security-checklist)
7. [Frontend Architecture](#7-frontend-architecture)
8. [Frontend Security Checklist](#8-frontend-security-checklist)
9. [AI Workflow, CI & Git Hygiene](#9-ai-workflow-ci--git-hygiene)
10. [Postman & Testing Guide](#10-postman--testing-guide)
11. [Master Prompts](#11-master-prompts)
12. [Domain Add-On Prompts](#12-domain-add-on-prompts)
13. [Document Maintenance](#13-document-maintenance)

1. Tech Stack

> Replace ``<YOUR_DOMAIN_UI_LIB>`` with whatever fits your industry (e.g. FullCalendar for scheduling, Chart.js for analytics, react-map-gl for mapping, etc.)

| Layer | Choice | Versions | Notes |
|---|-------------------------------|------------------------------|--------------|
| ----- ----- ----- | | | |
| ----- | | | |
| Runtime | Node.js | Web search current | |
| Active LTS – never hardcode an LTS number from docs Use nodejs.org + search | | | |
| Framework | Express | Search + `npm show` | |
| latest stable Major lines change – verify | | | |
| Language | TypeScript | Search + `npm show | |
| typescript` | | | |
| Strict mode enabled | | | |
| Database | MongoDB Atlas | – | Cloud-hosted |
| | | | |
| ODM | Mongoose | Search + `npm show` | |
| Match driver / Atlas docs | | | |
| Validation | Zod | Search + `npm show` | |
| Env + request bodies | | | |
| Auth | Google OAuth + Email/Password | – | Passport.js |
| strategies | | | |
| JWT | jsonwebtoken | Search + `npm show` + | |
| advisory Access + refresh tokens | | | |
| Password hashing | bcryptjs | Search + `npm show` | |
| Zero native deps | | | |
| Security headers | helmet | Search + `npm show` + | |
| advisory Tune CSP for your SPA/CDN | | | |
| CORS | cors | Search + `npm show` | |
| Explicit origin allowlist | | | |
| Rate limiting | express-rate-limit | Search + `npm show` | |
| Per-route limits | | | |
| NoSQL sanitize | express-mongo-sanitize | Search + `npm show` | |
| Prevent injection | | | |
| Real-time | socket.io | Search + `npm show` | |
| Room-based events | | | |
| Job scheduling | node-cron | Search + `npm show` | |
| Background tasks | | | |
| Email | nodemailer | Search + `npm show` | |
| SMTP / Resend / SendGrid | | | |

| | | | |
|--------------------------------------|--|------------------------------|---------|
| Logging | winston | Search + `npm show` | |
| Structured production logs | | | |
| Monitoring | @sentry/node | Search + `npm show` + | |
| Sentry docs** Error tracking | | | |
| Frontend | React + Vite | Search + `npm show` | |
| each SPA | | | |
| Routing | react-router-dom | Search + `npm show` | |
| Protected routes | | | |
| Server state | TanStack Query | Search + `npm show` | |
| Caching + retries | | | |
| HTTP client | Axios | Search + `npm show` | |
| Interceptors for refresh | | | |
| Forms | React Hook Form + Zod | Search + `npm show` | |
| each Validated forms | | | |
| HTML sanitize | DOMPurify | Search + `npm show` + | |
| advisory** Frontend XSS prevention | | | |
| Domain UI | ` <your_domain_ui_lib>`</your_domain_ui_lib> | - | |
| Industry-specific | | | |
| Deploy: Frontend | Vercel | - | Set env |
| vars in dashboard | | | |
| Deploy: Backend | Render / Railway | - | Set env |
| vars in dashboard | | | |
| Deploy: DB | MongoDB Atlas | - | IP |
| allowlist required | | | |

Token lifetime standard:

- Access JWT: `15 minutes` - sent in `Authorization: Bearer` header
- Refresh token: `7 days` - stored in `HttpOnly; Secure; SameSite=Strict` cookie

2. Repository Structure

...

```
ProjectRoot/
├─ backend/
│   └─ src/
│       └─ server.ts           # Entry: DB connect → cron start → listen
│       └─ app.ts             # Express app: middleware stack + route
└─ mounts
    └─ config/
        └─ env.ts             # Zod-validated env - crash on startup if
                                misconfigured
```

```

| | | └─ db.ts # mongoose.connect + disconnect
| | | └─ passport.ts # Google OAuth strategy (only if env vars
present)
| | └─ middleware/
| | | └─ requireAuth.ts # JWT verify → attach req.user
| | | └─ validate.ts # Zod schema factory → 400 on failure
| | | └─ roleGuard.ts # Resource-level role enforcement
| | | └─ errorHandler.ts # Central error → standard JSON response
| | └─ modules/ # One folder per domain resource
| | | └─ auth/
| | | | └─ auth.routes.ts
| | | | └─ auth.controller.ts
| | | | └─ auth.service.ts
| | | | └─ auth.schema.ts # Zod schemas for this module
| | | └─ users/
| | | | └─ user.routes.ts
| | | | └─ user.controller.ts
| | | | └─ user.service.ts
| | | | └─ user.model.ts
| | | | └─ user.schema.ts
| | | └─ <resource>/ # Repeat for each domain resource
| | └─ services/
| | | └─ email.service.ts # Nodemailer wrapper
| | | └─ cron.service.ts # node-cron background jobs
| | └─ sockets/
| | | └─ index.ts # Socket.io auth guard + room setup
| | └─ utils/
| | | └─ jwt.ts # signAccess, signRefresh, verify
| | | └─ encryption.ts # AES-256-GCM encrypt/decrypt
| | | └─ ownershipCheck.ts # Assert resource belongs to req.user
| | | └─ tokenCompare.ts # crypto.timingSafeEqual wrapper
| | └─ types/
| | | └─ express.d.ts # Augment Express Request with req.user
└─ postman/
| | └─ collection.json # Postman collection (committed)
| | └─ environment.json # Postman env template (no real secrets)
└─ .env.example # Committed template with dummy values
└─ .gitignore
└─ package.json
└─ tsconfig.json # Strict mode required

└─ frontend/
| | └─ src/
| | | └─ main.tsx
| | | └─ App.tsx # Route definitions

```

```

|   |   |   └─ lib/
|   |   |       └─ env.ts           # VITE_* Zod validation
|   |   |           └─ api/
|   |   |               └─ client.ts # Axios base + withCredentials +
interceptors
|   |   |           └─ refreshClient.ts # Separate instance for refresh (no loop)
|   |   |   └─ auth/
|   |   |       └─ AuthProvider.tsx    # Context: user, login, logout
|   |   |           └─ tokenStore.ts   # In-memory token (never localStorage)
|   |   |   └─ components/
|   |   |       └─ RequireAuth.tsx     # Route guard
|   |   |           └─ ErrorBoundary.tsx # Catch render errors
|   |   |   └─ features/
|   |   |       └─ <resource>/
|   |   |           └─ api.ts          # TanStack Query hooks
|   |   |               └─ components/
|   |   |                   └─ types.ts
|   |   └─ pages/
|   └─ .env.example
|   └─ .gitignore
|   └─ package.json
└─ docs/
    └─ PROJECT_BLUEPRINT.md           # This file
    └─ BACKEND_PLANNING.md            # Detailed planning doc
...

**Rules - never break these:**

- Never commit `.env`, `node_modules`, `dist/`, or `build/`
- Always commit `.env.example` with placeholder values and comments
- One source of truth for env validation: `src/config/env.ts` (backend),
  `src/lib/env.ts` (frontend)
- TypeScript strict mode always on - no `any` unless explicitly typed and commented

### `.gitignore` - dependencies, env files, secrets, and build output

**Commit:** source code, `package.json` / lockfiles, `.env.example`, Postman
templates (no real secrets), `README`, and `docs/`.

**Never commit:** real secrets, `node_modules`, or generated bundles.

Use a **root** `.gitignore` (or matching `backend/.gitignore` +
`frontend/.gitignore`) so **all** of the following stay out of git:

```

```

```gitignore
Dependencies (never commit - reinstall with npm ci)
node_modules/

Environment & secrets - NEVER commit (only .env.example is allowed)
.env
.env.*
!.env.example

Private keys and common secret filenames
*.pem
*.key
id_rsa
id_ed25519
*.p12
*.pfx

Build output
dist/
build/
out/
*.tsbuildinfo

Logs & coverage
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
coverage/
.nyc_output/

OS / editor noise (add .vscode/ or .idea/ only if your team agrees not to share
them)
.DS_Store
Thumbs.db
```

**Rules:**

- **`node_modules/`** - must be ignored everywhere it exists (root, `backend/`,
`frontend/`). Lockfile (`package-lock.json` or `pnpm-lock.yaml`) is committed
for reproducible installs.

- **`.env`, `.env.local`, `.env.production`, `.env.development`** - all covered by
`.env.*` with the `!.env.example` exception. Do not add "override" files that
contain secrets with a pattern that would be committed.

```

```
- **Verify before first push:** `git status` must not list `.env` or
`node_modules`. Use `git check-ignore -v path` if unsure.

**AI / IDE indexing:** Keep secrets out of model context – see [Section
9](#9-ai-workflow-ci--git-hygiene) (`.cursorignore` and copy-paste rules).

---

## 3. Environment Variables

### Backend `.env.example`

```env
— Server —————
NODE_ENV=development
PORT=5000

— Database —————
MONGODB_URI=mongodb://localhost:27017/your_db_name
Production: mongodb+srv://<user>:<pass>@cluster.mongodb.net/your_db

— JWT —————
Generate: node -e
"console.log(require('crypto').randomBytes(64).toString('hex'))"
JWT_ACCESS_SECRET=replace_with_64_char_hex
JWT_REFRESH_SECRET=replace_with_different_64_char_hex
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d

— Google OAuth (optional – remove if not using) —————
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
GOOGLE_CALLBACK_URL=http://localhost:5000/api/v1/auth/google/callback

— Frontend URL —————
CLIENT_URL=http://localhost:5173
Production: https://yourdomain.com

— CORS (comma-separated for multiple origins) —————
CORS_ORIGINS=http://localhost:5173,http://localhost:3000

— Encryption (for storing sensitive third-party tokens) —————
Generate: openssl rand -hex 32
ENCRYPTION_KEY=replace_with_64_char_hex
```



```

— Email (Nodemailer) —————
SMTP_HOST=smtp.resend.com
SMTP_PORT=465
SMTP_USER=resend
SMTP_PASS=your_smtp_api_key
EMAIL_FROM=noreply@yourdomain.com

— Push Notifications (optional) —————
Generate: npx web-push generate-vapid-keys
VAPID_PUBLIC_KEY=your_vapid_public_key
VAPID_PRIVATE_KEY=your_vapid_private_key
VAPID_EMAIL=mailto:admin@yourdomain.com

— Monitoring —————
SENTRY_DSN=https://your_sentry_dsn_here
...

Frontend `.env.example`

```.env
# — API —————
VITE_API_BASE_URL=http://localhost:5000

# — Push Notifications (public key only) —————
VITE_VAPID_PUBLIC_KEY=your_vapid_public_key

# — Feature Flags (optional) —————
VITE_ENABLE_REAL_TIME=true
...

### Env Validation Pattern (Backend)

```typescript
// src/config/env.ts
import { z } from 'zod';

const envSchema = z.object({
 NODE_ENV: z.enum(['development', 'production', 'test']),
 PORT: z.string().transform(Number).default('5000'),
 MONGODB_URI: z.string().url(),
 JWT_ACCESS_SECRET: z.string().min(32),
 JWT_REFRESH_SECRET: z.string().min(32),
 JWT_ACCESS_EXPIRES_IN: z.string().default('15m'),
 JWT_REFRESH_EXPIRES_IN: z.string().default('7d'),
 CLIENT_URL: z.string().url(),

```

```

 CORS_ORIGINS: z.string(),
 ENCRYPTION_KEY: z.string().length(64),
 // Optional - only required if feature is enabled
 GOOGLE_CLIENT_ID: z.string().optional(),
 GOOGLE_CLIENT_SECRET: z.string().optional(),
 SMTP_HOST: z.string().optional(),
 SENTRY_DSN: z.string().optional(),
 });

const parsed = envSchema.safeParse(process.env);
if (!parsed.success) {
 console.error('❌ Invalid environment variables:',
 parsed.error.flatten().fieldErrors);
 process.exit(1); // Crash immediately - never run with bad config
}

export const env = parsed.data;
```

---

## 4. Backend Architecture

### Middleware Stack Order (app.ts)

Every request passes through this chain in order. Order matters.

```typescript
// src/app.ts
import * as Sentry from '@sentry/node';
import express from 'express';
import helmet from 'helmet';
import cors from 'cors';
import mongoSanitize from 'express-mongo-sanitize';
import rateLimit from 'express-rate-limit';
import morgan from 'morgan';

// 1. Sentry request handler (must be first)
app.use(Sentry.Handlers.requestHandler());

// 2. Security headers
app.use(helmet());

// 3. CORS - explicit origin list only, never '*' with credentials
app.use(cors({

```

```

origin: env.CORS_ORIGINS.split(','),
credentials: true,
methods: ['GET', 'POST', 'PATCH', 'DELETE', 'OPTIONS'],
));

// 4. Body parsing
app.use(express.json({ limit: '10kb' })); // Limit body size
app.use(express.urlencoded({ extended: true }));

// 5. NoSQL injection prevention - strips $ and . from req.body/query/params
app.use(mongoSanitize());

// 6. HTTP request logging (dev only)
if (env.NODE_ENV === 'development') app.use(morgan('dev'));

// 7. Global rate limit
const globalLimiter = rateLimit({ windowMs: 60_000, max: 100, standardHeaders:
true, legacyHeaders: false });
app.use('/api', globalLimiter);

// 8. Strict auth-route rate limit
const authLimiter = rateLimit({ windowMs: 60_000, max: 10, skipSuccessfulRequests:
true });
app.use('/api/v1/auth', authLimiter);

// 9. Routes
app.use('/api/v1', router);

// 10. Sentry error handler (before your error handler)
app.use(Sentry.Handlers.errorHandler());

// 11. Central error handler (always last)
app.use(errorHandler);
```


### Health & readiness (production hosts)



Platforms (Render, Railway, Kubernetes, load balancers) expect HTTP endpoints that **do not require auth**.



- **`GET /health` (liveness)** - returns `200` if the process is up (no DB call). Cheap and fast.
- **`GET /ready` (readiness, optional)** - returns `200` only if dependencies are usable (e.g. `mongoose.connection.readyState === 1`); otherwise `503`.

```

Mount these **before** or **outside** heavy global middleware if needed so a bad config does not block the health check from answering (or keep them minimal and dependency-free on `/health`` only).

Register **before** the global `/api`` rate limiter if your provider hammers the health path, or **exclude** `/health`` and `/ready`` from strict rate limits.

Standard Error Response Shape

All errors across the entire API must follow this shape. Define it once, use it everywhere.

```
```typescript
// All error responses:
{
 "success": false,
 "error": {
 "code": "ERROR_CODE", // Machine-readable constant
 "message": "Human readable", // Display to user
 "fields": { // Only present on VALIDATION_ERROR
 "email": ["Invalid email address"],
 "password": ["Must be at least 8 characters"]
 }
 }
}

// All success responses:
{
 "success": true,
 "data": { ... } // Resource or array
}

// Paginated success:
{
 "success": true,
 "data": [...],
 "pagination": {
 "total": 100,
 "page": 1,
 "limit": 20,
 "totalPages": 5
 }
}
...
```
```

Ownership Check Helper

Use this in every single controller that accesses a resource by ID. Never trust that a resource belongs to a user just because they're logged in.

```
```typescript
// src/utils/ownershipCheck.ts
import { Model, Types } from 'mongoose';

export async function assertOwnership<T>(
 ModelClass: Model<T>,
 resourceId: string,
 userId: string
): Promise<T> {
 const doc = await ModelClass.findOne({
 _id: new Types.ObjectId(resourceId),
 userId: new Types.ObjectId(userId)
 });
 if (!doc) {
 // Always 404, never 403 - don't confirm the resource exists
 const err = new Error('Resource not found') as any;
 err.statusCode = 404;
 err.code = 'NOT_FOUND';
 throw err;
 }
 return doc;
}

// Usage in any controller:
const item = await assertOwnership(Event, req.params.id, req.user._id);
```
```

Timing-Safe Token Comparison

```
```typescript
// src/utils/tokenCompare.ts
import crypto from 'crypto';

export function safeCompare(a: string, b: string): boolean {
 if (a.length !== b.length) return false;
 return crypto.timingSafeEqual(Buffer.from(a), Buffer.from(b));
}
```
```

5. API Documentation Standard

Every endpoint in your project must be documented in this format. This section defines the standard – fill it in for each module.

Template (copy for each endpoint)

...

METHOD /api/v1/<resource>/<action>

****Description:**** One sentence describing what this does.

****Auth required:**** Yes / No

****Minimum role:**** owner / admin / member / public

Request

Headers:

Authorization: Bearer <accessToken> (if auth required)

Content-Type: application/json

Path params:

:id – MongoDB ObjectId of the resource

Query params:

?page=1&limit=20 – pagination

?filter=value – filtering

Body:

```
{
  "field": "value",          // Required. Description.
  "optionalField": "val"    // Optional. Default: null. Description.
}
```

Response – 200 OK (GET, PATCH, DELETE)

```
{
  "success": true,
  "data": { ... }
}
```

Response – 201 Created (POST – new resource created)

```
{
  "success": true,
  "data": {
    "_id": "64a1b2c3d4e5f6a7b8c9d0e1",
    ...resource fields
  }
}
```

```

    }
}

#### Response - 400 Validation Error
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Validation failed",
    "fields": { "field": ["Error message"] }
  }
}

#### Response - 401 Unauthorized
{ "success": false, "error": { "code": "TOKEN_EXPIRED", "message": "Access token
expired" } }

#### Response - 403 Forbidden
{ "success": false, "error": { "code": "FORBIDDEN", "message": "Insufficient
permissions" } }

#### Response - 404 Not Found
{ "success": false, "error": { "code": "NOT_FOUND", "message": "Resource not
found" } }

#### Postman example
Method: POST
URL: {{baseUrl}}/api/v1/<resource>
Body (raw JSON):
{
  "field": "example value"
}
Pre-request script: (if needed)
Tests: pm.test("Status 200", () => pm.response.to.have.status(200));
      pm.environment.set("resourceId", pm.response.json().data._id);

#### Use cases
- Use case 1: When a manager creates X, they send this request.
- Use case 2: When a user wants to Y, the frontend calls this.

#### Business rules
- Rule 1: Field X must be unique per user.
- Rule 2: Only the owner can set field Y to value Z.
...

```

Error Codes Master Reference

| HTTP | Code | When to use |
|-------|-------------------------|---|
| ----- | ----- | ----- |
| 400 | `VALIDATION_ERROR` | Zod validation failed on body/query/params |
| 400 | `INVALID_REQUEST` | Logically invalid request (e.g. end before start) |
| 400 | `CONFIRM_TEXT_MISMATCH` | Destructive action confirmation text wrong |
| 401 | `UNAUTHORIZED` | No token provided |
| 401 | `TOKEN_EXPIRED` | Access token expired – client should refresh |
| 401 | `TOKEN_INVALID` | Token tampered or wrong secret |
| 401 | `REFRESH_TOKEN_INVALID` | Refresh token not found or already rotated |
| 403 | `FORBIDDEN` | Authenticated but wrong role |
| 403 | `CANNOT_SELF_DEMOTE` | User tried to remove their own admin/owner role |
| 404 | `NOT_FOUND` | Resource not found or doesn't belong to this user |
| 409 | `CONFLICT` | Duplicate resource (unique field already exists) |
| 429 | `RATE_LIMIT_EXCEEDED` | Too many requests in window |
| 500 | `INTERNAL_ERROR` | Unhandled server error – check Sentry |

6. Backend Security Checklist

Run through this before every production deployment.

Environment & Configuration

- All secrets in `.env` – zero secrets hardcoded in source
- `.env` and all `.env.\*` in `.gitignore`
- Env validated with Zod at startup – app crashes on bad config
- `JWT\_ACCESS\_SECRET` and `JWT\_REFRESH\_SECRET` are different, each ≥64 chars
- `ENCRYPTION\_KEY` is 32 random bytes (64 hex chars) – stored in env only
- MongoDB connection string uses a restricted DB user (not root)
- MongoDB Atlas IP allowlist configured – only your server IPs

Authentication

- Access tokens short-lived (15min), refresh tokens long-lived (7 days)
- Access token sent in `Authorization` header only – never in URL
- Refresh token stored in `HttpOnly; Secure; SameSite=Strict` cookie
- Refresh tokens hashed with bcrypt before storing in DB
- Refresh token rotation implemented – new token on every `/auth/refresh` call
- Reused refresh token triggers full revocation for that user (breach signal)
- `crypto.timingSafeEqual()` used for all token comparisons – no `===`
- OAuth `state` parameter validated to prevent CSRF on callback


```
- Account lockout after N failed login attempts (5 attempts → 15 min lock)
- Session management: users can view and revoke active sessions

### API Security

- `helmet()` enabled – secure HTTP headers (CSP, XSS, clickjacking)
- `cors()` configured with explicit origin list – never `*` with credentials
- `express-mongo-sanitize()` in middleware chain – NoSQL injection prevention
- `express-rate-limit` on all routes – strict limits on auth endpoints
- `express.json({ limit: '10kb' })` – prevent payload size attacks
- Zod validation on every route that accepts body/query/params
- `assertOwnership()` used in every controller – no IDOR vulnerabilities
- All MongoDB queries filter by `userId` – cross-user data access impossible
- Soft-delete pattern – `isDeleted: false` filter on all queries
- Public endpoints (booking pages, invite accept) rate limited separately

### Data & Privacy

- Passwords hashed with bcrypt (cost factor ≥ 10) – never stored plain
- Sensitive third-party tokens (Google OAuth tokens) encrypted with AES-256-GCM
- Invite tokens generated with `crypto.randomBytes(32)` – not sequential
- Invite errors return generic messages (prevent email enumeration)
- `DELETE /users/me` requires confirmation text – prevents accidental deletion
- `GET /users/me/export` endpoint exists (GDPR data portability)
- TTL indexes on ephemeral collections (invites, soft-deleted records, login attempts)
- Text search index only on non-sensitive fields

### Infrastructure

- `GET /health` (liveness) implemented; `GET /ready` with DB check if your host requires readiness
- Health/readiness routes excluded from aggressive rate limits if the platform polls them often
- Winston structured logging in production – **no passwords, access/refresh tokens, API keys, full card numbers, or raw PII** in log messages; redact or hash identifiers where logs are needed for debugging
- Sentry error monitoring configured and tested – scrub sensitive data in `beforeSend` / `denyUrls` as needed
- `npm audit` clean – zero high/critical vulnerabilities
- Dependabot or Snyk monitoring dependencies
- `npm ci` used in CI (not `npm install`)
- API versioning prefix (`/api/v1/`) in place
- Socket.io connections validated with JWT before joining any room
- HTTPS enforced in production – no HTTP
```

7. Frontend Architecture

Axios Client Setup

```typescript

// src/lib/api/client.ts

import axios from 'axios';

import { tokenStore } from '../auth/tokenStore';

import { refreshClient } from './refreshClient';

export const client = axios.create({

baseUrl: import.meta.env.VITE\_API\_BASE\_URL + '/api/v1',

withCredentials: true, // Send HttpOnly refresh cookie

timeout: 10\_000,

});

// Attach access token to every request

client.interceptors.request.use((config) => {

const token = tokenStore.get();

if (token) config.headers.Authorization = `Bearer \${token}`;

return config;

});

// Single-flight refresh on 401 - prevents multiple simultaneous refresh calls

let isRefreshing = false;

let refreshQueue: Array<{ resolve: Function; reject: Function }> = [];

client.interceptors.response.use(

(res) => res,

async (error) => {

const original = error.config;

if (error.response?.status === 401 &&

error.response?.data?.error?.code === 'TOKEN\_EXPIRED' &&

!original.\_retry) {

original.\_retry = true;

if (isRefreshing) {

return new Promise((resolve, reject) => {

refreshQueue.push({ resolve, reject });

}).then(token => {

original.headers.Authorization = `Bearer \${token}`;

return client(original);

});

```

 }
 isRefreshing = true;
 try {
 const { data } = await refreshClient.post('/auth/refresh');
 tokenStore.set(data.data.accessToken);
 refreshQueue.forEach(p => p.resolve(data.data.accessToken));
 refreshQueue = [];
 original.headers.Authorization = `Bearer ${data.data.accessToken}`;
 return client(original);
 } catch {
 refreshQueue.forEach(p => p.reject());
 refreshQueue = [];
 tokenStore.clear();
 window.location.href = '/login';
 return Promise.reject(error);
 } finally {
 isRefreshing = false;
 }
 }
 return Promise.reject(error);
}
);
...

```typescript
// src/lib/api/refreshClient.ts - separate instance, no interceptors (prevents
infinite loop)
import axios from 'axios';
export const refreshClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL + '/api/v1',
  withCredentials: true,
});
...

```typescript
// src/auth/tokenStore.ts - access token in MEMORY only, never localStorage
let _token: string | null = null;
export const tokenStore = {
 get: () => _token,
 set: (t: string) => { _token = t; },
 clear: () => { _token = null; },
};
...

Auth Provider Pattern

```

```

```typescript
// src/auth/AuthProvider.tsx
import { createContext, useContext, useEffect, useState } from 'react';
import { client } from '../lib/api/client';
import { tokenStore } from '../tokenStore';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  // Bootstrap: try to restore session on app load
  useEffect(() => {
    (async () => {
      try {
        // Try refresh first - if refresh cookie exists, we get a new access token
        const { data: refresh } = await refreshClient.post('/auth/refresh');
        tokenStore.set(refresh.data.accessToken);
        // Then fetch user profile
        const { data: me } = await client.get('/users/me');
        setUser(me.data);
      } catch {
        // No valid session - stay logged out
      } finally {
        setLoading(false);
      }
    })();
  }, []);

  return (
    <AuthContext.Provider value={{ user, loading, setUser }}>
      {loading ? <FullPageSpinner /> : children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

Route Protection

```typescript
// src/components/RequireAuth.tsx

```

```

import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '../auth/AuthProvider';

export function RequireAuth({ children }: { children: React.ReactNode }) {
  const { user, loading } = useAuth();
  const location = useLocation();
  if (loading) return <FullPageSpinner />;
  if (!user) return <Navigate to="/login" state={{ from: location }} replace />;
  return <>{children}</>;
}
...

### TanStack Query Pattern

```typescript
// src/features/<resource>/api.ts
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { client } from '../../lib/api/client';

// Read
export function useResources(params) {
 return useQuery({
 queryKey: ['resources', params],
 queryFn: () => client.get('/resources', { params }).then(r => r.data.data),
 staleTime: 1000 * 60 * 5, // 5 minutes
 });
}

// Create
export function useCreateResource() {
 const qc = useQueryClient();
 return useMutation({
 mutationFn: (body) => client.post('/resources', body).then(r => r.data.data),
 onSuccess: () => qc.invalidateQueries({ queryKey: ['resources'] }),
 });
}

// Update
export function useUpdateResource() {
 const qc = useQueryClient();
 return useMutation({
 mutationFn: ({ id, ...body }) => client.patch(`/resources/${id}`, body).then(r
=> r.data.data),
 onSuccess: () => qc.invalidateQueries({ queryKey: ['resources'] }),
 });
}

```

```

}

// Delete
export function useDeleteResource() {
 const qc = useQueryClient();
 return useMutation({
 mutationFn: (id) => client.delete(`/resources/${id}`),
 onSuccess: () => qc.invalidateQueries({ queryKey: ['resources'] }),
 });
}
...

8. Frontend Security Checklist

Token & Session Security

- Access token stored in memory only (`tokenStore.ts`) – never `localStorage` or `sessionStorage`
- Refresh token is HttpOnly cookie – JavaScript cannot read it
- Single-flight refresh queue implemented – no duplicate refresh calls
- On failed refresh → clear token store + redirect to login
- OAuth callback route reads token from URL param then immediately clears the URL (`history.replaceState`)
- No sensitive data (tokens, passwords, PII) logged to console in production

XSS Prevention

- Never use `dangerouslySetInnerHTML` without sanitizing with `DOMPurify` first
- All user-generated content rendered as React text nodes (JSX `{}`) – not raw HTML
- URLs from user input validated before use in `href` or `src` – only `http:` and `https:`
- No direct `innerHTML` or `document.write()` usage
- Content Security Policy (CSP) header set by backend helmet – verified working
- `DOMPurify.sanitize()` used whenever rendering rich text or markdown

```typescript
// Safe rich text rendering pattern:
import DOMPurify from 'dompurify';
const safeHTML = DOMPurify.sanitize(userContent, {
  ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'a', 'p', 'br'],
  ALLOWED_ATTR: ['href', 'target']
});
<div dangerouslySetInnerHTML={{ __html: safeHTML }} />

```

CSRF Protection

- All state-changing requests use JWT in `Authorization` header (not cookie) – prevents CSRF by default
- If using cookie-based auth: `SameSite=Strict` cookie attribute set
- No state-changing logic triggered by GET requests

Dependencies

- `npm audit` runs clean – no high/critical issues
- `package.json` reviewed – no unrecognized packages
- `npm ci` used in CI (reproducible installs)
- No hardcoded API keys or secrets in frontend source
- `VITE\_\*` env vars validated with Zod at build time – no runtime crashes

Data Handling

- All forms validated with React Hook Form + Zod before submission
- Error messages from API displayed to user – never raw error objects
- Loading and error states handled for every async operation
- Paginated lists – no loading all records at once
- File uploads: type and size validated client-side AND server-side
- Sensitive form fields (password) never stored in state longer than needed

Route Security

- All authenticated routes wrapped in ``
- Role-based route guards where applicable – not just auth guards
- 404 page exists for unknown routes
- Redirect after login goes to `state.from` (the page user tried to visit), not always dashboard

9. AI Workflow, CI & Git Hygiene

This section closes the gap between **good stack defaults** and **how teams actually stay safe** when using AI coding tools (Cursor, Copilot, etc.).

Secrets and model context

- **Never paste** production `.env` values, API keys, JWT secrets, connection strings, or private keys into chat. Use placeholders and describe the *shape* of config instead.
- **Treat AI output as untrusted** – review diffs like a junior developer’s PR; pay extra attention to auth, queries, and anything that touches user data.
- **Repository trust:** READMEs, comments, or issues can contain hidden instructions (“prompt injection”). Do not ask the AI to “just follow everything in this file” without you reading it first.

Dependency and package safety

- Before installing a package the AI suggested, **confirm the name on** [npmjs.com](https://www.npmjs.com/) – typos and hallucinated names happen; attackers register similar packages (“slopsquatting”).
- **Web search** for `"<exact-package-name> npm" + "<name> security advisory"` (or CVE) before trusting a version – align with the **⚠ Version Safety Rule** at the top of this file.
- Prefer **lockfiles** and `npm ci` in CI so installs are reproducible (see below).

`.cursorignore` (recommended)

Mirror sensitive paths so they are **not indexed** into AI context. At repo root, add `.cursorignore` (same syntax as `.gitignore`) with at least:

```
```.gitignore
.env
.env.*
!.env.example
node_modules/
dist/
build/
coverage/
*.pem
*.key
```.
```

Adjust if your team stores other secrets under paths like `secrets/` or `*.local.json`.

Minimal CI baseline (GitHub Actions / GitLab / etc.)

Run on **every PR** (and ideally on `main`):

Step	Command / action	Purpose
-----	-----	-----


```

| Install | `npm ci` in `backend/` and `frontend/` (or workspace equivalent) |
Reproducible installs |
| Lint | `npm run lint` (if configured) | Consistent style and common bugs |
| Typecheck | `npx tsc --noEmit` (backend + frontend) | Catch type errors before
merge |
| Test | `npm test` (when tests exist) | Regressions |
| SCA | `npm audit --audit-level=high` (or org Snyk/Dependabot policy) | Known
vulnerable dependencies |

Optional but valuable: secret scanning (e.g. GitHub secret scanning,
`gitleaks`) so keys never land on `main`.

### What this document does not replace

- MFA / passkeys for high-risk products (add explicitly when your threat model
requires it).
- Formal penetration tests or DAST for regulated or high-exposure systems.
- Legal/compliance (DPAs, subprocessors, retention policies) – beyond the GDPR
export checklist elsewhere.

---

## 10. Postman & Testing Guide

### Collection Setup

1. Import `backend/postman/collection.json` into Postman
2. Import `backend/postman/environment.json` and set:
  - `baseUrl`: `http://localhost:5000`
  - Leave token variables empty – pre-request scripts fill them

### Run Order (adapt for your domain)

...

Step 1: POST /auth/register → creates user account
Step 2: POST /auth/login → sets refresh cookie + returns access token
Step 3: GET /users/me → verifies auth works
Step 4: POST /<primary-resource> → create your main resource
Step 5: GET /<primary-resource> → list resources
Step 6: PATCH /<primary-resource>/:id → update resource
Step 7: DELETE /<primary-resource>/:id → delete resource
Step 8: POST /auth/refresh → verify token refresh works
Step 9: POST /auth/logout → verify cookie cleared
Step 10: GET /users/me → verify 401 after logout
...

```

```
### Pre-Request Script Template
```

```
```javascript
```

```
// Add to collection root pre-request script:
```

```
// Automatically refreshes access token if expired
```

```
const tokenExpiry = pm.environment.get('tokenExpiry');
```

```
const now = Date.now();
```

```
if (tokenExpiry && now >= parseInt(tokenExpiry)) {
```

```
 pm.sendRequest({
```

```
 url: pm.environment.get('baseUrl') + '/api/v1/auth/refresh',
```

```
 method: 'POST',
```

```
 header: { 'Content-Type': 'application/json' },
```

```
 }, function (err, res) {
```

```
 if (!err && res.code === 200) {
```

```
 const token = res.json().data.accessToken;
```

```
 pm.environment.set('accessToken', token);
```

```
 pm.environment.set('tokenExpiry', Date.now() + 14 * 60 * 1000); // 14
```

```
min
```

```
 }
```

```
 });
```

```
}
```

```
```
```

```
### Test Script Template
```

```
```javascript
```

```
// Add to each endpoint's Tests tab:
```

```
// 1. Assert status
```

```
pm.test("Status is 200", () => pm.response.to.have.status(200));
```

```
// 2. Assert response shape
```

```
pm.test("Response has success: true", () => {
```

```
 const json = pm.response.json();
```

```
 pm.expect(json.success).to.be.true;
```

```
 pm.expect(json.data).to.exist;
```

```
});
```

```
// 3. Save IDs for subsequent requests
```

```
const json = pm.response.json();
```

```
if (json.data?._id) {
```

```
 pm.environment.set('resourceId', json.data._id);
```

```

}

// 4. Save access token after login
if (json.data?.accessToken) {
 pm.environment.set('accessToken', json.data.accessToken);
 pm.environment.set('tokenExpiry', Date.now() + 14 * 60 * 1000);
}
...

Testing All Response Scenarios

For every endpoint, test at minimum:

| Test | How to trigger |
| ----- | ----- |
| Happy path (200/201) | Valid request |
| Validation error (400) | Missing required field |
| Unauthorized (401) | Remove Authorization header |
| Token expired (401) | Use expired token |
| Forbidden (403) | Use account with wrong role |
| Not found (404) | Use non-existent or other user's ID |
| Rate limited (429) | Send 11+ requests in 1 minute on auth route |
| Conflict (409) | Create duplicate unique resource |

11. Master Prompts

11.1 Full-Stack Bootstrap Prompt

Copy, fill in the ``, and paste into your AI assistant:

...

Build a production-ready full-stack app with TypeScript.

Industry: <YOUR_INDUSTRY>
Primary resources: <RESOURCE_1>, <RESOURCE_2>, <RESOURCE_3>
(e.g. "healthcare scheduling: appointments, patients, doctors")

--- Backend ---
- Monorepo: backend/ (Express 5 + Mongoose + TypeScript) and frontend/ (Vite + React + TypeScript)

```

- Before writing any package.json: **\*\*web search\*\*** each dependency for latest stable + security advisories, then confirm with `npm show <pkg> version` – never copy versions from this doc
- Structure: src/config/env.ts (Zod, crash on bad config), db.ts, app.ts, server.ts  
Modules under src/modules/<resource>/ with model + routes + controller + service + schema
- Auth: Google OAuth + email/password; JWT access (15min) in JSON response; refresh (7d) in HttpOnly; Secure; SameSite=Strict cookie; rotation with hashed refresh token  
in User model; logout clears cookie + DB entry; Google OAuth only when env vars exist
- Security middleware (in this order): Sentry request handler, helmet, cors (explicit origins),  
express.json({limit:'10kb'}), express-mongo-sanitize, rate-limit, requireAuth, validate, roleGuard
- All errors follow: { success: false, error: { code, message, fields? } }
- All success responses follow: { success: true, data: {...} }
- Use assertOwnership() helper in every controller – always 404 never 403 on wrong user
- crypto.timingSafeEqual() for all token comparisons
- Zod validation on every route body/query/params
- Winston structured logging in production – never log passwords, tokens, API keys, or raw PII; Sentry with sensitive data scrubbing
- node-cron for background jobs (if needed for the domain)
- Socket.io for real-time updates with JWT auth guard on connection
- AES-256-GCM encryption for any sensitive third-party tokens stored in DB
- GET /health (liveness) and GET /ready (readiness, Mongo connected); exclude from strict rate limits if the host polls often

## — Frontend —

- Validate VITE\_API\_BASE\_URL and all VITE\_\* vars with Zod in src/lib/env.ts
- Axios instance with withCredentials:true; single-flight refresh interceptor on 401  
using a separate refreshClient (no interceptors) to prevent infinite loops
- Access token in memory only (tokenStore.ts) – never localStorage or sessionStorage
- AuthProvider bootstraps session on load via POST /auth/refresh then GET /users/me
- RequireAuth wrapper for all protected routes
- TanStack Query for all server state; invalidate on mutation
- React Hook Form + Zod for all forms
- DOMPurify for any user-generated HTML content
- Error boundary at root level
- Domain-specific UI: <YOUR\_DOMAIN\_UI\_LIBRARY>

## — Deliverables —

```
1. .env.example for both backend and frontend with comments
2. Root .gitignore (and package roots if split) - node_modules/, .env*, keys,
dist/build; only .env.example for secrets template; lockfile committed
3. .cursorignore at repo root mirroring sensitive paths (.env*, node_modules, dist,
keys) per Section 9
4. README with setup instructions (install, seed, run) + minimal CI instructions
(npm ci, typecheck, test, npm audit) or a .github/workflows CI file
5. Postman collection + environment JSON under backend/postman/
6. TypeScript strict mode tsconfig.json for backend (and frontend)
7. Full API documentation for each endpoint following the template in
PROJECT_BLUEPRINT.md
...

```

```
11.2 Backend-Only Prompt

```

```
...
```

Build a production-ready Express 5 + TypeScript + MongoDB backend.

Domain: <YOUR\_DOMAIN>

Resources: <RESOURCE\_1>, <RESOURCE\_2>

Before writing package.json: web search (latest + CVE/advisory) for each dependency, confirm with `npm show`; do not use hardcoded versions from any blueprint.

Requirements:

- Zod-validated env at startup - crash if misconfigured
- Email/password + optional Google OAuth (Passport, only when env vars set)
- JWT access (15min) + refresh (7d) in HttpOnly cookie with rotation and breach detection
- Security: helmet, cors (explicit list), express-mongo-sanitize, express-rate-limit
- assertOwnership() in every controller - 404 not 403 for wrong user
- crypto.timingSafeEqual() for token comparisons
- bcryptjs for passwords (cost 10+) and hashed refresh tokens
- Zod on every route; consistent { success, error: {code, message, fields} } shape
- Winston logging in production - no passwords, tokens, or raw PII in logs; Sentry error monitoring with scrubbing
- GET /health + GET /ready (DB connectivity for ready)
- Soft delete pattern (isDeleted + deletedAt) on primary resources
- Full-text search on primary resource (MongoDB text index)
- Pagination on all list endpoints (?page&limit)
- Socket.io with JWT auth guard for real-time updates

Deliver: .env.example, README, Postman collection, full endpoint documentation per resource

```

11.3 Security Hardening Only Prompt

```

Audit and harden this Node.js + Express + MongoDB API for production.

Check and implement if missing:

1. express-mongo-sanitize – NoSQL injection
2. express-rate-limit – per-route (strict on auth, general on API, public on booking/invite)
3. helmet() – all security headers
4. CORS with explicit origin list from env – no wildcard
5. express.json({ limit: '10kb' }) – body size limit
6. All DB queries filtered by userId – no IDOR
7. assertOwnership() helper – 404 not 403 on wrong user
8. crypto.timingSafeEqual() – all token comparisons
9. Refresh token: bcrypt hash in DB, rotate on use, revoke all on reuse
10. Account lockout: track failed attempts, lock after 5 for 15 min (TTL index)
11. Generic error messages on invite/auth – no email enumeration
12. AES-256-GCM for sensitive tokens stored in DB
13. Zod validation on every route body/query/params
14. TTL indexes: soft-deleted records, expired invites, login attempts
15. Winston + Sentry in production
16. npm audit – fix all high/critical before deploying
17. Socket.io JWT guard on connection before any room join
18. GDPR: GET /users/me/export endpoint
19. GET /health (liveness) + GET /ready (readiness); tune rate limits so health checks are not throttled
20. Log redaction – no secrets or full PII in Winston; configure Sentry beforeSend scrubbing
21. .gitignore verified – .env\*, node\_modules, dist, keys never tracked; only .env.example committed
22. Optional: .cursorignore for AI indexing (Section 9)

```

11.4 Frontend Security Hardening Prompt

```

Audit and harden this React + TypeScript + Axios frontend for production.

Check and implement if missing:

1. Access token in memory only (tokenStore) – remove any localStorage usage

2. Single-flight refresh interceptor – no duplicate refresh calls
  3. Separate refreshClient without interceptors – prevent refresh loops
  4. DOMPurify on all dangerouslySetInnerHTML usage
  5. URL validation before href/src – only http: and https:
  6. All forms validated with React Hook Form + Zod before submit
  7. Error boundary at root level
  8. RequireAuth on all protected routes
  9. OAuth callback clears token from URL immediately after reading
  10. No console.log of tokens or sensitive data (strip in production)
  11. npm audit clean
  12. VITE\_\* env vars validated with Zod at build time
  13. Loading + error states for every async operation
  14. Paginated lists – no unbounded data fetching
  15. File uploads validated client-side (type + size) before sending
- ...

---

## ## 12. Domain Add-On Prompts

These are short add-ons to append to the master prompt for specific industries. Replace the generic UI library reference with the domain-specific one.

### ### Scheduling / Calendar

...

Domain UI: FullCalendar React (dayGridMonth + timeGridWeek + timeGridDay + interaction plugin)

Resources: events (4 types: event/task/out\_of\_office/appointment), calendars, reminders, teams

Special features:

- RRule (RFC 5545) for recurrence; rrule.js on both server and client
- Event edit modes: "this" / "this\_and\_following" / "all"
- date-fns-tz for UTC ↔ user timezone conversion; all dates stored as UTC
- Team availability: free/busy query with calendarVisibility (full/busy\_only/none)
- Slot-finder: interval-merge algorithm to find common free windows
- Web Push notifications via Service Worker + VAPID keys
- node-cron: poll reminders every 60s, fire push/email on triggerAt <= now

...

### ### E-Commerce / Marketplace

...

Domain UI: Product grids, cart, checkout flow

Resources: products, orders, carts, reviews, categories

Special features:

- Stripe payment integration (server-side intent creation – never handle card data directly)
  - Order state machine: pending → confirmed → shipped → delivered → refunded
  - Inventory tracking with optimistic locking (prevent oversell)
  - Image uploads via Cloudinary (client gets signed upload URL from server)
  - Price stored in smallest currency unit (cents) – never floats
- ...

### ### Project Management / CRM

...

Domain UI: Kanban board (react-beautiful-dnd or @dnd-kit/core)

Resources: projects, tasks, boards, columns, comments, attachments

Special features:

- Drag-and-drop position ordering (use fractional indexing – never integer positions)
  - @mentions in comments (notify mentioned users via email/push)
  - File attachments via pre-signed S3 URLs
  - Activity log per project (who changed what, when)
  - Webhook delivery for external integrations
- ...

### ### Analytics / Dashboard

...

Domain UI: Chart.js or Recharts for charts; react-grid-layout for dashboard widgets

Resources: dashboards, widgets, data\_sources, reports

Special features:

- Aggregation pipelines in MongoDB for metric computation
  - Date range filtering with timezone-aware bucketing
  - Cache computed metrics in Redis (or MongoDB with TTL) – don't compute on every request
  - CSV/Excel export endpoints
  - Scheduled report delivery via email (node-cron + nodemailer)
- ...

### ### Social / Community

...

Domain UI: Infinite scroll lists, comment threads, notification center

Resources: posts, comments, likes, follows, notifications

Special features:

- Cursor-based pagination (not page/offset – works with real-time inserts)



```

- Fan-out on write vs fan-out on read for notification delivery (choose based on
scale)
- Content moderation: soft-delete + report flag + admin review queue
- Rate limit post creation separately from reads
- Full-text search with MongoDB Atlas Search or Elasticsearch
...

13. Document Maintenance

This document reflects processes that stay valid over time; dependency
numbers are never authoritative here.

When to update this file:

- When you add a new integration (payments, file storage, email provider, maps) →
add its env vars to both .env.example files and to the env Zod schema, then add a
domain add-on prompt in Section 12
- When you change auth strategy (e.g. add MFA, switch to Passkeys) → update Section
6 checklist and Section 11 prompts
- When you change AI/CI/git hygiene practices → update Section 9
- When a CVE is published for a dependency you use → rotate secrets if
affected, upgrade to a patched release (verify via search + `npm audit`),
document the incident in your own changelog – do not add long-lived version pins to
this file
- When deploying to a new hosting provider → re-verify cookie `Secure` flag, CORS
origins, and HTTPS enforcement

Security review triggers:

- Any change to cookie domain, `SameSite`, or `Secure` attributes
- Any change to CORS `origin` list
- Any change to OAuth redirect URLs
- Any new public endpoint (no auth) – add to rate-limit config

AI assistant instruction:

When using this document as context, follow the ⚠ Version Safety Rule at the
top: web search for each dependency (`"<pkg> npm latest version"`, "`<pkg>
CVE" / advisory`), confirm with `npm show <pkg> version` when the shell is
available, state what you verified in your answer, and never copy semver
literals from this file into `package.json`.

```

*\*Industry-agnostic. Adapt the domain add-ons in Section 12 for your use case.\**